<!-- به نام خدا -->

مهندسی اینترنت، فصل ۱: پروتکل‌ها

وضعیت (state) و هویت

جلسهٔ ۳: کوکی و SOP و CORS و CSRF؛ پروتکل stateless چگونه کاربر را می‌شناسد؟

parse · HSTS · DNS · اتصال · TLS ◌ · HTTP ●

محمد بصیرزاده

دانشگاه شهید چمران اهواز، دانشکدهٔ مهندسی، پاییز ۱۴۰۵

راز ماندگاری HTTP چیست؟

پروتکل HTTP در ۱۳۷۰، پیش از تولد بیشتر شما، فقط یک سند HTML می‌گرفت؛ امروز ویدیو پخش می‌کند، پول جابه‌جا می‌کند و agentها را به ابزار وصل می‌کند. رازش چیست؟

الف) سرعت بالا

ب) فیلدهای header

ج) رمزنگاری داخلی

د) قالب باینری

راز ماندگاری HTTP چیست؟

پروتکل HTTP در ۱۳۷۰، پیش از تولد بیشتر شما، فقط یک سند HTML می‌گرفت؛ امروز ویدیو پخش می‌کند، پول جابه‌جا می‌کند و agentها را به ابزار وصل می‌کند. رازش چیست؟

الف) سرعت بالا	ب) فیلدهای header	ج) رمزنگاری داخلی	د) قالب باینری

پاسخ: ب) گیرندهٔ پیام header ناشناخته را نادیده می‌گیرد (جلسهٔ ۲)؛ پس هر قابلیت تازه، از cache تا CORS، با یک فیلد تازه آمد.

> **HTTP با header تازه رشد کرد، نه با تغییر خودش؛ state و هویت هم همین‌طور آمدند.**

وضعیت (state) و هویت در HTTP چه جایگاهی دارند؟



- HTTP پروتکلی stateless است: سرور درخواست‌های قبلی را به یاد نمی‌آورد
- چرا؟ سرور ساده می‌ماند و هزاران اتصال هم‌زمان را پاسخ می‌دهد؛ وگرنه پس از هر خرابی، state دو طرف باید دوباره سازگار شود (Kurose و Ross، بخش ۲.۲.۴)
- پس state را برنامه نگه می‌دارد، با دو header تازه: `Set-Cookie` و `Cookie`

> **پروتکل stateless است و برنامه stateful؛ کوکی پل میان این دو است.**

بخش ۱ از ۶

کوکی: حافظهٔ برنامه روی HTTP

تا اینجا: در تمرین ۲ دیدیم IP و headerهای HTTP برای شناختن کاربر کافی نیستند: دو دانشجو پشت یک مودم خوابگاه یک نفر شمرده شدند.

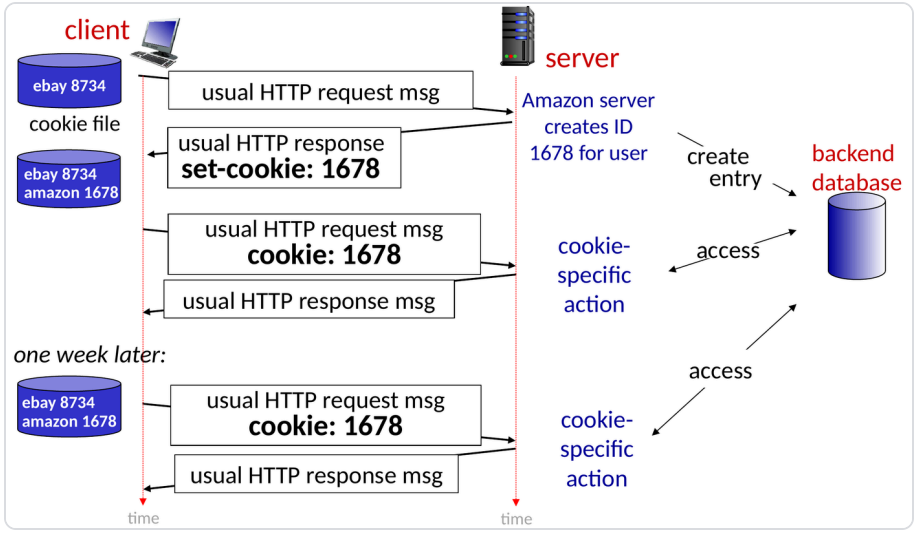
سؤال این بخش

سرور و مرورگر با چه ترفندی می‌توانند درخواست بعدی را به درخواست قبلی مرتبط کنند؟ و چه کسانی به این ترفند دسترسی دارند؟

03-state-identity.todo

- [x] کوکی ▶
- [ ] SOP و CORS
- [ ] CSRF و SameSite
- [ ] احراز هویت
- [ ] چالش‌ها
- [ ] جمع‌بندی

کوکی چگونه کار می‌کند و از چه اجزایی تشکیل می‌شود؟



حفظ state با استفاده از کوکی

منبع: Kurose & Ross, Figure 2.10 · author slides

کوکی چهار جزء دارد (Kurose و Ross، بخش ۲.۲.۴):

- `Set-Cookie` در پاسخ سرور
- `Cookie` در **هر درخواست بعدی**
- فایل کوکی در مرورگر
- پایگاه دادهٔ سرور

کاربرد: نشست کاربر، سبد خرید، پیشنهاد

> **پیام‌ها همچنان مستقل‌اند؛ شناسه‌ای که در هر درخواست تکرار می‌شود آن‌ها را به هم مرتبط می‌کند.**

هر ویژگی کوکی چه کاربردی دارد؟

ویژگی در `Set-Cookie`	پرسش
`Domain` و `Path`	به زیردامنه‌ها هم برود؟ برای کدام مسیرها؟
`Expires` و `Max-Age`	تا چه زمانی بماند؟ بی این دو، با بستن مرورگر پاک می‌شود
`Secure`	فقط روی HTTPS برود؟ (جلسهٔ ۴)
`HttpOnly`	از JavaScript صفحه **پنهان بماند**؟
`SameSite`	همراه درخواستی که سایت دیگری آغاز کرده هم برود؟ (بخش ۳)

ویژگی‌ها دستور به مرورگرند و در `Cookie` برنمی‌گردند (MDN، پیش‌نویس 6265bis).

> **هر ویژگی یک قید است: کجا، تا چه زمانی، با چه شرطی، و برای چه کسی.**

کوکی سامانهٔ درس برای کدام زیردامنه‌ها فرستاده می‌شود؟

پاسخ ورود به `lms.scu.ac.ir` این فیلد را دارد:

```
Set-Cookie: sid=7f3a9c; Domain=scu.ac.ir; Secure
```

الف) فقط `lms`

ب) `lms` و `www`

ج) همهٔ زیردامنه‌ها

د) همهٔ `ac.ir`

کوکی سامانهٔ درس برای کدام زیردامنه‌ها فرستاده می‌شود؟

پاسخ ورود به `lms.scu.ac.ir` این فیلد را دارد:

```
Set-Cookie: sid=7f3a9c; Domain=scu.ac.ir; Secure
```

الف) فقط `lms`

ب) `lms` و `www`

ج) همهٔ زیردامنه‌ها

د) همهٔ `ac.ir`

پاسخ: ج) با `Domain`، کوکی به `scu.ac.ir` و همهٔ زیردامنه‌هایش می‌رود، حتی وبلاگی ناامن روی `blog.scu.ac.ir`؛ بی آن، فقط به خود `lms.scu.ac.ir`. پیشوند `__Host-` در نام کوکی آن را به همین host قفل می‌کند: زیردامنهٔ دیگری نمی‌تواند کوکی هم‌نامی برای این host بنویسد.

> **`Domain` کوکی را به میزبان‌های بیشتری می‌فرستد، نه کمتر؛ فقط در صورت نیاز آن را بنویسید.**

خودتان امتحان کنید: یک کوکی روی دو پنجره

- در سایتی که حساب دارید وارد شوید و در DevTools، زبانهٔ Application و بخش Cookies، کوکی نشست را پیدا کنید
- همان سایت را در پنجرهٔ ناشناس باز کنید؛ وارد نشده‌اید
- در همان بخش DevTools پنجرهٔ ناشناس، کوکی‌ای با همان نام و مقدار بسازید و صفحه را تازه کنید
- حالا در پنجرهٔ اصلی از حساب خارج شوید و پنجرهٔ ناشناس را دوباره تازه کنید

فقط روی حساب خودتان. اگر سرور نشست را در سمت خود نگه دارد، با خروج، کوکی کپی‌شده هم باطل می‌شود.

> **سرور فقط مقدار کوکی را می‌بیند، نه مرورگری که آن را فرستاده است.**

تا اینجا دیدیم سایتی که خودتان باز کرده‌اید می‌تواند کوکی بسازد؛ اما صفحه‌ها معمولاً تصویر و اسکریپت را از سایت‌های دیگری هم بارگیری می‌کنند.

چه کسی شما را از سایتی به سایت دیگر دنبال می‌کند؟

- اگر `news.example` تصویری از `ads.example` بگیرد، کوکی `ads.example` هم همراهش می‌رود: **کوکی شخص ثالث** (third-party cookie)، در برابر کوکی شخص اولِ سایتی که باز کرده‌اید
- این کوکی در هر سایتی که آن تصویر را دارد یکسان است، حتی اگر تصویر یک پیکسل نامرئی باشد؛ قانون GDPR اروپا چنین کوکی‌ای را دادهٔ شخصی می‌داند
- مرورگر Safari آن را مسدود می‌کند و Firefox برای هر سایت جدا نگه می‌دارد؛ Chrome در دی ۱۳۹۸ وعدهٔ حذفش را داد و در اردیبهشت ۱۴۰۴ از آن برگشت

> **کوکی به مقصد درخواست مربوط است، نه لزوماً به صفحه‌ای که کاربر باز کرده است.**

بخش ۲ از ۶

محدودیت دسترسی به پاسخ: SOP و CORS

تا اینجا: مرورگر کوکی را خودکار برای مقصد درخواست می‌فرستد، حتی اگر درخواست از صفحهٔ سایت دیگری آغاز شده باشد.

سؤال این بخش

اگر هر صفحه‌ای بتواند همراه کوکی‌های شما به سایتی که در آن وارد شده‌اید درخواست بفرستد، چه چیزی مانع می‌شود که پاسخ را بخواند؟

03-state-identity.todo

[x] کوکی
[>] SOP و CORS
[ ] CSRF و SameSite
[ ] احراز هویت
[ ] چالش‌ها
[ ] جمع‌بندی

SOP چه چیزی را محدود می‌کند؟

Same-Origin Policy قاعده‌ای در مرورگر است که دسترسی اسکریپت یک origin به داده‌های origin دیگر را محدود می‌کند. هر origin از سه بخش تشکیل شده است: scheme و host و port (جلسهٔ ۱).

کاری که صفحهٔ `evil.example` با `bank.example` می‌کند	مجاز است؟
فرستادن درخواست با `<form>` یا `<img>` یا `fetch`	بله
جاسازی تصویر و اسکریپت و iframe آن	بله؛ ولی اسکریپت محتوایش را نمی‌بیند
خواندن پاسخ با JavaScript	نه، مگر سرور اجازه دهد
خواندن کوکی و DOM آن origin	نه

> **SOP جلوی خواندن پاسخ را می‌گیرد، نه لزوماً جلوی ارسال درخواست را.**

اسکریپت سایت دیگر موجودی شما را می‌خواهد؛ چه می‌شود؟

در `bank.example` وارد شده‌اید. اسکریپت `evil.example` اجرا می‌کند:

```
fetch('https://bank.example/balance')
```

الف) فرستاده نمی‌شود

ب) فقط می‌رسد

ج) سرور رد می‌کند

د) خوانده می‌شود

اسکریپت سایت دیگر موجودی شما را می‌خواهد؛ چه می‌شود؟

در `bank.example` وارد شده‌اید. اسکریپت `evil.example` اجرا می‌کند:

```
fetch('https://bank.example/balance')
```

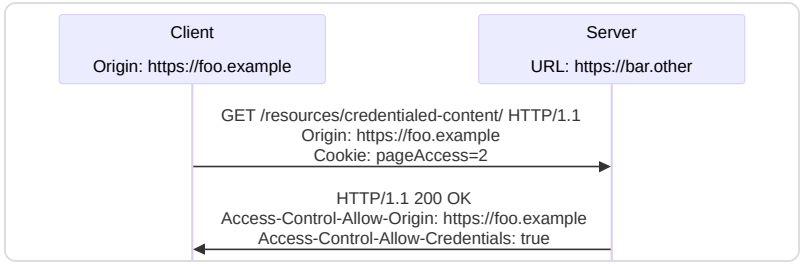
الف) فرستاده نمی‌شود	ب) فقط می‌رسد	ج) سرور رد می‌کند	د) خوانده می‌شود

پاسخ: ب) درخواست می‌رود و سرور آن را پردازش می‌کند؛ اما پاسخ اجازهٔ CORS ندارد، پس مرورگر آن را به اسکریپت نمی‌دهد و خطا برمی‌گرداند. همراه رفتن کوکی را در بخش ۳ می‌بینیم.

> **درخواست رسید و اجرا شد؛ مرورگر فقط خواندن پاسخ را بست.**

CORS چگونه به origin دیگر اجازهٔ خواندن می‌دهد؟

نام کامل: Cross-Origin Resource Sharing، یعنی اشتراک منابع میان originها

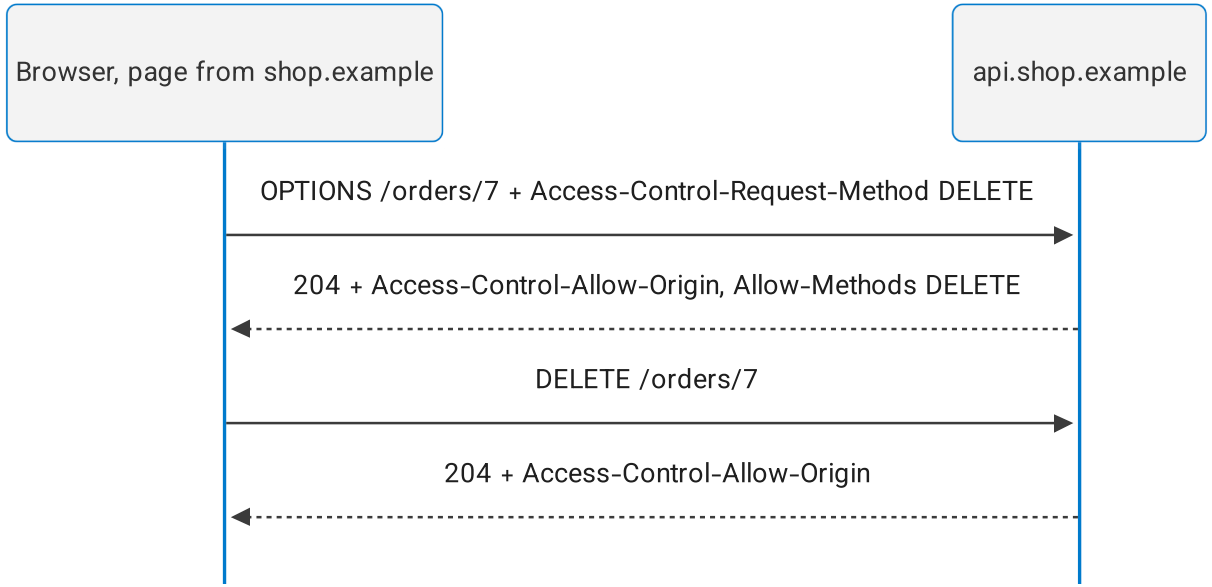


منبع: MDN, Mozilla Contributors · CC BY-SA 2.5

- مرورگر فیلد `Origin` را به درخواست اضافه می‌کند؛ اگر پاسخ با `Access-Control-Allow-Origin` همان origin را تأیید کند، **پاسخ را به اسکریپت می‌دهد**
- با کوکی، `Access-Control-Allow-Credentials: true` هم لازم است و `*` پذیرفته نمی‌شود

> **CORS دیوار تازه‌ای نیست؛ دری است که سرور در دیوار SOP باز می‌کند.**

چرا مرورگر پیش از بعضی درخواست‌ها preflight می‌فرستد؟



- اگر درخواستی از آنچه فرم HTML می‌تواند بسازد فراتر برود، مرورگر ابتدا با `OPTIONS` **مجوز لازم را بررسی می‌کند**؛ به این درخواست مقدماتی preflight می‌گویند

> **با preflight، سرور پیش از رسیدن درخواست غیرمعمول اعلام می‌کند آن را می‌پذیرد یا نه.**

کدام درخواست‌ها به preflight نیاز دارند؟

صفحه‌ای از `evil.example` این چهار درخواست را به `api.shop.example` می‌فرستند:

```
1 GET /items
2 POST /pay (HTML form body)
3 POST /pay (JSON body)
4 DELETE /items/7
```

الف) هیچ‌کدام

ب) فقط ۴

ج) ۳ و ۴

د) ۲ و ۳ و ۴

کدام درخواست‌ها به preflight نیاز دارند؟

صفحه‌ای از `evil.example` این چهار درخواست را به `api.shop.example` می‌فرستند:

```
1 GET /items
2 POST /pay (HTML form body)
3 POST /pay (JSON body)
4 DELETE /items/7
```

الف) هیچ‌کدام

ب) فقط ۴

ج) ۳ و ۴

د) ۲ و ۳ و ۴

پاسخ: ج) ۱ و ۲ را فرم HTML هم می‌سازد، پس بی preflight می‌روند؛ بدنهٔ JSON و متد `DELETE` از فرم برنمی‌آیند.

> **درخواستِ ساختنی با فرم HTML بی preflight می‌رسد؛ کنترل دسترسی با سرور است.**

آیا CORS مانع ارسال یا اجرای درخواست می‌شود؟

سؤال اگر origin درخواست مجاز نباشد، آیا سرور باز هم درخواست را اجرا می‌کند؟

مشاهده پاسخ سرور به درخواست‌های GET و POST از `https://evil.example` و تغییر احتمالی موجودی حساب را بررسی کنید

تصمیم CORS جایگزین کنترل دسترسی در سرور نیست؛ سرور باید برای هر عملیات، هویت درخواست‌کننده و مجاز بودن آن را بررسی کند.

```
U=http://127.0.0.1:8773/api
curl -si -H 'Origin: https://evil.example' $U/balance
curl -si -X POST -H 'Origin: https://evil.example' $U/transfer
curl -s $U/balance
```

سرور محلی در `labs/03-state-identity/cors` است. curl برخلاف مرورگر SOP را اعمال نمی‌کند؛ در این آزمایش، فیلد `Origin` را خودمان به درخواست اضافه می‌کنیم.

نتیجهٔ آزمایش: سرور چه کرد و مرورگر چه واکنشی نشان داد؟

درخواست	پاسخ سرور	مرورگر با آن چه می‌کرد
GET از `app.example` **(مجاز)**	`200` با `Access-Control-Allow-Origin`	پاسخ را به اسکریپت می‌دهد
GET از `evil.example`	`200` با همان بدنه، بدون آن فیلد	پاسخ را به اسکریپت نمی‌دهد
preflight برای **DELETE** از `evil.example`	`204` بدون `Access-Control-Allow-Methods`	DELETE را نمی‌فرستد
POST از `evil.example`	`200` ؛ **موجودی از ۱۰۰۰ به ۹۰۰ رسید**	پاسخ را نمی‌دهد؛ ولی کار انجام شده

- همهٔ پاسخ‌ها فیلد `Vary: Origin` را دارند تا cache پاسخ یک origin را به origin دیگری تحویل ندهد (جلسهٔ ۵)

بخش ۳ از ۶

جعل درخواست (CSRF) و ویژگی SameSite

تا اینجا: خواندن پاسخ محدود شده است، اما ارسال درخواست همچنان ممکن است و مرورگر هم کوکی را به‌طور خودکار همراه آن می‌فرستد.

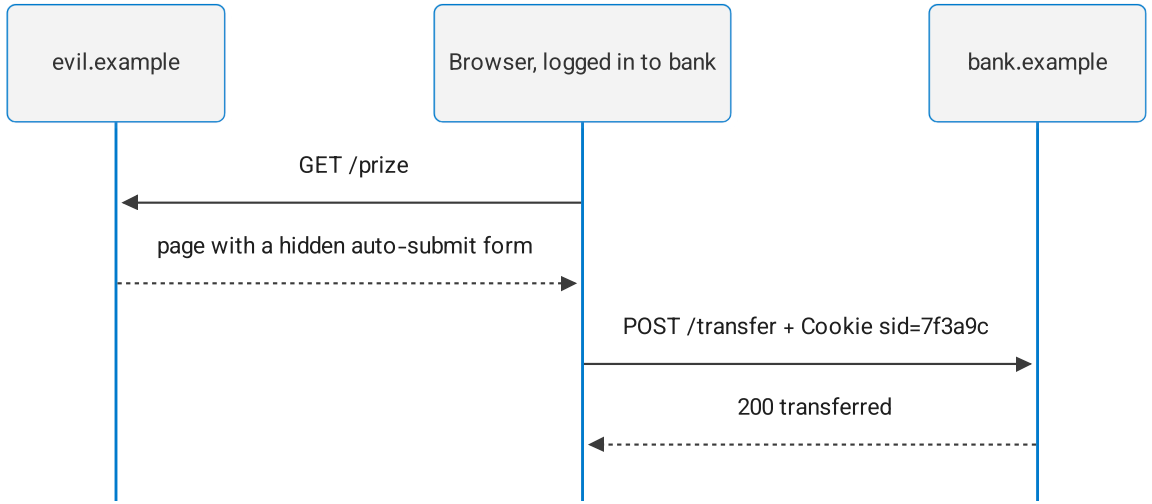
سؤال این بخش

اگر صفحه‌ای دیگر به نام شما درخواست بفرستد، سرور چگونه تشخیص می‌دهد که درخواست واقعاً از طرف شماست؟

03-state-identity.todo

[x] کوکی
[x] SOP و CORS
▶ [>] CSRF و SameSite
[ ] احراز هویت
[ ] چالش‌ها
[ ] جمع‌بندی

CSRF چیست و چگونه کار می‌کند؟



- در CSRF (Cross-Site Request Forgery) صفحهٔ مهاجم به سایتی درخواست می‌فرستد که کاربر در آن وارد شده، و مرورگر کوکی را خودکار همراهش می‌کند. لازم نیست مهاجم پاسخ را ببیند؛ **اثر درخواست** کافی است

> **سرور کوکی معتبر را می‌بیند، اما از روی آن نمی‌فهمد کدام صفحه درخواست را فرستاده است.**

نمونهٔ واقعی: انتقال پول با باز کردن یک صفحهٔ وب

- مهر ۱۳۸۷: دو پژوهشگر دانشگاه پرینستون نشان دادند در بانک اینترنتی ING Direct، صفحه‌ای بیگانه می‌تواند به نام کاربرِ واردشده **حساب تازه بسازد و پول منتقل کند** (Zeller و Felten)
- همان گزارش نشان می‌داد که در YouTube تقریباً هر عملی را می‌شد به نام کاربر انجام داد و در وب‌سایت New York Times نیز نشانی ایمیل کاربران افشا می‌شد
- در هیچ‌یک از این حمله‌ها لازم نبود رمز عبور دزدیده شود یا پاسخ درخواست خوانده شود

> **اگر سرور برای مجاز دانستن عملیات فقط به کوکی تکیه کند، هر صفحه‌ای در وب می‌تواند به نام کاربر عمل کند.**

SameSite تعیین می‌کند کوکی چه زمانی ارسال شود

درخواست به `bank.example`	`Strict`	`Lax`	`None`
از صفحه‌ای در خود `bank.example`	✓	✓	✓
کلیک روی لینک در سایت دیگر (GET سطح بالا)	✗	✓	✓
فرم POST یا `fetch` یا `<img>` در سایت دیگر	✗	✗	✓

- مقدار `None` فقط همراه `Secure` پذیرفته می‌شود (MDN)
- Chrome از بهمن ۱۳۹۸ کوکی بدون `SameSite` را `Lax` حساب می‌کند؛ پیش‌فرض Firefox و Safari متفاوت است

> **`Lax` جلوی فرم و `fetch` سایت دیگر را می‌گیرد، ولی جلوی لینک معمولی را نه.**

کوکی Strict از زیردامنه‌ای دیگر هم ارسال می‌شود؟

کوکی `bank.example.com` ویژگی `SameSite=Strict` دارد. مهاجم در `blog.example.com` فرمی می‌گذارد که به `bank.example.com/transfer` ارسال می‌شود.

الف) نه؛ origin دیگر

ب) بله؛ هم‌سایت‌اند

ج) فقط با GET

د) فقط با `Domain`

کوکی Strict از زیردامنه‌ای دیگر هم ارسال می‌شود؟

کوکی `bank.example.com` ویژگی `SameSite=Strict` دارد. مهاجم در `blog.example.com` فرمی می‌گذارد که به `bank.example.com/transfer` ارسال می‌شود.

الف) نه؛ origin دیگر

ب) بله؛ هم‌سایت‌اند

ج) فقط با GET

د) فقط با `Domain`

پاسخ: ب) `SameSite` مرز site را می‌سنجد، یعنی scheme و دامنهٔ قابل‌ثبت (جلسهٔ ۱)؛ پورت هم جزء آن نیست.

> **`SameSite` مرز site را می‌کشد، نه origin را.**

آزمایش CSRF در بانک آزمایشی

سؤال آیا فرم پنهان صفحهٔ مهاجم با همهٔ مقادیر `SameSite` موفق می‌شود؟

مشاهده موجودی حساب، پیش و پس از باز کردن صفحهٔ مهاجم: یک بار از `127.0.0.1:8772` (سایت دیگر) و یک بار از `localhost:8772` (همان سایت، پورت دیگر)

تصمیم `SameSite` را صریح بنویسید و تنها به آن تکیه نکنید

```
node bank/bank.mjs     # bank:  http://localhost:8771
node bank/evil.mjs     # evil:  http://127.0.0.1:8772/post   http://localhost:8772/post
```

در مرورگر وارد بانک آزمایشی شوید، مقدار `SameSite` را از فهرست فرم انتخاب کنید، صفحهٔ مهاجم را باز کنید و سپس به بانک برگردید. کد آزمایش در `labs/03-state-identity/bank` است.

نتیجهٔ آزمایش: کوکی همراه کدام درخواست ارسال شد؟

کوکی نشست	POST از سایت دیگر	POST از localhost:8772	<img> از سایت دیگر	لینک از سایت دیگر
بدون SameSite	رفت	رفت	نرفت	رفت
Lax	نرفت	رفت	نرفت	رفت
Strict	نرفت	رفت	نرفت	نرفت
None; Secure	رفت	رفت	رفت	رفت

کوکی بدون SameSite دقیقاً Lax نیست: Chromium تا دو دقیقه پس از ساخت آن، POST سطح بالا از سایت دیگر را هم مجاز می‌داند. ستون دوم همان site با پورتی دیگر است. اجرا با Chromium 141 و bank/auto.mjs .

انتخاب SameSite بین Lax و Strict، همراه با چه دفاعی؟

لایهٔ دفاع	چه می‌کند	کجا کم می‌آورد
`SameSite=Lax` **صریح**	کوکی با فرم سایت دیگر نمی‌رود	زیردامنهٔ هم‌سایت
`SameSite=Strict`	با لینک سایت دیگر هم نمی‌رود	کاربرِ آمده از ایمیل، «واردنشده» است
token ضد CSRF در فرم	صفحهٔ بیگانه **آن را نمی‌خواند**	فرمی که token را جا بیندازد
بررسی `Sec-Fetch-Site`	سرور مبدأ درخواست را می‌بیند	کلاینت قدیمی آن را نمی‌فرستد

این چهار لایه را راهنمای پیشگیری از CSRF در OWASP پیشنهاد می‌کند؛ OWASP بنیادی غیرانتفاعی است که مرجع‌های آزاد امنیت وب را منتشر می‌کند.

> `Lax` **برای نشست، یک دفاع دوم در سرور، و هیچ تغییری با GET.**

بخش ۴ از ۶

مروری بر احراز هویت

تا اینجا: کوکی می‌گوید «من همان کلاینت قبلی‌ام»، اما به‌خودی‌خود هویت آن کلاینت را مشخص نمی‌کند.

سؤال این بخش

سرور در نخستین درخواست چگونه هویت شما را تشخیص دهد؟ و کلاینتی که مرورگر نیست، چگونه هویت خود را ارائه کند؟

03-state-identity.todo

- [x] کوکی
- [x] SOP و CORS
- [x] CSRF و SameSite
- [>] احراز هویت ▶
- [ ] چالش‌ها
- [ ] جمع‌بندی

سرور چگونه از کلاینت احراز هویت می‌خواهد؟

- سرور با `401` و فیلد `WWW-Authenticate` می‌گوید چه طرحی می‌پذیرد و کلاینت مدرک را در فیلد `Authorization` می‌فرستد (RFC 9110 §11)
- ساده‌ترین طرح، `Basic` ، نام کاربری و رمز عبور را با Base64 کد می‌کند؛ این **رمزنگاری نیست** و هر بیننده‌ای آن را برمی‌گرداند
- چالشش: رمز عبور در هر درخواست جابه‌جا می‌شود و راهی برای «خروج» ندارد

```
GET /grades HTTP/1.1

HTTP/1.1 401 Unauthorized
WWW-Authenticate: Basic realm="lms"

GET /grades HTTP/1.1
Authorization: Basic YWxpOnNlY3JldA==
```

طرح `Basic` ساده است، اما رمز عبور را در هر درخواست می‌فرستد؛ طرح‌های بعدی همین را حل می‌کنند.

چرا به‌جای رمز عبور token می‌فرستیم؟

- کلاینت یک بار وارد می‌شود و token می‌گیرد؛ سپس در هر درخواست `Authorization: Bearer <token>` می‌فرستد (RFC 6750)
- token را می‌شود **محدود کرد**: برای یک سرویس، با عمر کوتاه، و قابل ابطال بی‌آنکه رمز عبور عوض شود
- واژهٔ bearer یعنی «دارنده»: هر کس token را داشته باشد از آن استفاده می‌کند؛ پس فقط روی HTTPS

> **مدرکی که در هر درخواست می‌رود باید محدود، کوتاه‌عمر و قابل ابطال باشد، نه خودِ رمز عبور.**

کوکی نشست یا Bearer token؟

	کوکی نشست	bearer token
چه کسی آن را به درخواست اضافه می‌کند	مرورگر، به‌طور خودکار	کد برنامه، به‌صورت صریح
CSRF	**خطر دارد**؛ دفاع بخش ۳	ندارد؛ صفحهٔ بیگانه token را ندارد
دزدی با اسکریپت تزریق‌شده (XSS)	با `HttpOnly` خوانده نمی‌شود	اگر JavaScript ببیندش، برده می‌شود
کلاینت غیرمرورگر، مثل curl و agent	دشوار	ساده

برای نشست مرورگر کوکی، و برای کلاینت برنامه‌ای و API معمولاً token.

در token چه بگذاریم: شناسهٔ تصادفی یا دادهٔ امضاشده؟

چالش: وقتی چند سرور پشت هم کار می‌کنند، هر کدام باید token را در یک پایگاه دادهٔ مشترک جستجو کند. JWT اطلاعات کاربر را همراه امضای سرور در خود token می‌گذارد تا هر سرور بی‌جستجو آن را بسنجد. جزئیات JWT در فصل بعد.

	شناسهٔ نشست	دادهٔ امضاشده (JWT)
state کجاست	در سرور؛ token فقط یک عدد تصادفی است	در خود token؛ سرور فقط امضا را می‌سنجد
خروج و ابطال	سرور رکورد را پاک می‌کند؛ **فوری**	تا لحظهٔ انقضا معتبر می‌ماند
هزینه	یک جستجو در هر درخواست	بی‌جستجو؛ ولی token بزرگ‌تر
خواندن محتوا	فقط سرور	هر دارنده؛ امضا فقط جلوی تغییر را می‌گیرد

> **هرچه بخش بیشتری از state را به کلاینت بسپارید، ابطال آن دشوارتر می‌شود.**

token را در مرورگر کجا نگه داریم؟

فرض کنید برنامهٔ تک‌صفحه‌ای شما پس از ورود یک token دریافت می‌کند، اما یکی از کتابخانه‌های npm که صفحه از آن استفاده می‌کند آلوده است.

الف) `localStorage`

ب) `sessionStorage`

ج) کوکی `HttpOnly`

د) در query نشانی

token را در مرورگر کجا نگه داریم؟

فرض کنید برنامهٔ تک‌صفحه‌ای شما پس از ورود یک token دریافت می‌کند، اما یکی از کتابخانه‌های npm که صفحه از آن استفاده می‌کند آلوده است.

الف)	ب)	ج) کوکی	د) در query نشانی
`localStorage`	`sessionStorage`	`HttpOnly`	

پاسخ: ج) اسکریپت آلوده هرچه را JavaScript ببیند می‌خواند و بیرون می‌فرستد؛ کوکی `HttpOnly` را نمی‌بیند، هرچند هنوز می‌تواند از همان صفحه درخواست بفرستد (OWASP).

> **هرچه اسکریپت صفحهٔ شما بتواند بخواند، اسکریپت آلودهٔ همان صفحه هم می‌تواند بخواند.**

OAuth: اعطای دسترسی بدون افشای رمز عبور

- مسئله: برنامه‌ای باید تقویم شما را بخواند، بی‌آنکه رمز عبور حسابتان را بداند
- در OAuth نزد «سرور مجوز» وارد می‌شوید و برنامه یک token **محدود** می‌گیرد: برای یک سرویس، با دامنهٔ دسترسی (scope) مشخص و عمر کوتاه؛ برنامه همان را با `Authorization: Bearer` به API می‌فرستد
- «ورود با Google» روی همین جریان ساخته شده و لایهٔ احراز هویت (OpenID Connect) را به آن افزوده است؛ جزئیات در فصل بعد

> **رمز عبور همه‌چیز را باز می‌کند؛ token فقط همان دری را که لازم است.**

بخش ۵ از ۶

چالش‌های امنیتی: سرقت یا جعل اعتبارنامه

تا اینجا: کوکی و SOP و CORS و CSRF و احراز هویت را دیدیم.

سؤال این بخش

این سازوکارها در عمل چگونه شکست خورده‌اند؟ و agentها چه تغییری در مسئله ایجاد می‌کنند؟

03-state-identity.todo

- [x] کوکی
- [x] SOP و CORS
- [x] CSRF و SameSite
- [x] احراز هویت
- [>] **چالش‌ها** ▶
- [ ] جمع‌بندی

یادآوری بخش ۱: سرور فقط مقدار کوکی را می‌بیند.

نمونهٔ واقعی: افشای نشست‌ها از طریق فایل پشتیبانی

- در مهر ۱۴۰۲، مهاجمان فایل‌های HAR مشتریان را از سامانهٔ پشتیبانی Okta، شرکتی فعال در حوزهٔ مدیریت هویت، به دست آوردند
- فایل HAR ضبطی از ترافیک مرورگر است و می‌تواند **کوکی‌ها و tokenهای نشست** را هم در خود داشته باشد. مهاجمان با استفاده از همین اطلاعات، بدون دانستن رمز عبور یا نیاز به عامل دوم، به نشست مدیران Cloudflare و چند شرکت دیگر دسترسی پیدا کردند (Cloudflare)
- راه‌های دفاع شامل کوتاه نگه‌داشتن عمر نشست و امکان ابطال آن در سرور است. همچنین از بهار ۱۴۰۵، Chrome در Windows نشست را به کلیدی در تراشهٔ امنیتی دستگاه پیوند می‌دهد (DBSC)

> **از نگاه سرور، هر کس کوکی معتبر نشست را داشته باشد، همان کاربر است.**

از بخش ۲: اجازهٔ CORS را سرور به origin مشخصی می‌دهد.

نمونهٔ واقعی: پیکربندی نادرست CORS و پذیرش هر Origin

- بعضی سرورها مقدار `Origin` درخواست را بی‌بررسی در `Access-Control-Allow-Origin` **بازتاب می‌دهند** و `Access-Control-Allow-Credentials: true` هم می‌فرستند
- پژوهشی در مهر ۱۳۹۵ نشان داد با این خطا صفحه‌ای بیگانه کلید API کاربران یک صرافی بیت‌کوین را می‌خواند، و با آن کلید می‌شد دارایی را منتقل کرد
- صرافی دیگری فقط آغاز origin را می‌سنجید و `https://btc.net.evil.net` را هم می‌پذیرفت؛ همان خطای مقایسهٔ رشته در تابع تمرین ۱

> **`Origin` را با فهرستی دقیق مقایسه کنید؛ نه بازتابش دهید، نه با پیشوند بسنجید.**

یادآوری بخش‌های ۲ و ۳: SOP و SameSite دسترسی صفحه‌های بیگانه را محدود می‌کنند.

ارتباط با هوش مصنوعی: agentی که با کوکی‌های شما وب‌گردی می‌کند

- مرورگرهای agentدار به‌جای شما می‌خوانند و کلیک می‌کنند، **با همهٔ نشست‌های واردشدهٔ شما**
- در مرداد ۱۴۰۴، دستوری پنهان در یک نظر Reddit کافی بود تا agent مرورگر Comet نشانی ایمیل و رمز یک‌بارمصرف کاربر را از Gmail بیرون بکشد (Brave)
- سازوکارهای SOP و CORS و `SameSite` صفحهٔ بیگانه را محدود می‌کنند، اما agent در همان مرورگر و به نام کاربر کار می‌کند

> **دفاع مرورگر جلوی صفحهٔ بیگانه را می‌گیرد، نه agentی که به نام شما کار می‌کند.**

یادآوری بخش ۴: کد 401، فیلد WWW-Authenticate و token محدود.

ارتباط با هوش مصنوعی: agent با مجوز چه کسی کار می‌کند؟

```
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Bearer resource_metadata=
    "https://mcp.example.com/.well-known/oauth-protected-resource"
```

- در MCP، پروتکل اتصال agent به ابزار (جلسهٔ ۷)، سرور ابزار هم `401` می‌فرستد و نشانی سندی را می‌دهد که می‌گوید token را از کجا بگیرد (RFC 9728)
- token باید **فقط برای همان سرور** صادر شده باشد و در `Authorization` بیاید، نه در query

پروتکل تازه سازوکار تازه نخواست؛ همان `401` دههٔ ۱۹۹۰ کافی بود.

یادآوری بخش ۴: رمز عبور مدرکی با دسترسی گسترده است.

چرا به passkey نیاز داریم؟

- رمز عبور را کاربر در اختیار سایت می‌گذارد؛ اگر صفحه‌ای جعلی شبیه سایت اصلی باشد، ممکن است کاربر آن را همان‌جا وارد کند (homograph در جلسهٔ ۱)
- در passkey کلید خصوصی در دستگاه می‌ماند و مرورگر هر امضا را به **origin همان صفحه** گره می‌زند؛ امضای سایت جعلی به کار سایت اصلی نمی‌آید (WebAuthn)
- سرور کلید عمومی را نگه می‌دارد و پس از ورود موفق، همچنان می‌تواند مانند قبل کوکی نشست بسازد

> **در passkey مرورگر origin را می‌سنجد، نه چشم کاربر.**

بخش ۶ از ۶

جمع‌بندی

تا اینجا: در این جلسه سازوکار کوکی، محدودیت دسترسی به پاسخ، جعل درخواست و احراز هویت را بررسی کردیم.

سؤال این بخش

برای ایجاد نشست در سامانهٔ درس دانشگاه، یک فیلد `Set-Cookie` مناسب چه شکلی دارد؟

03-state-identity.todo

- [x] کوکی
- [x] SOP و CORS
- [x] CSRF و SameSite
- [x] احراز هویت
- [x] چالش‌ها
- [>] **جمع‌بندی** ▶

کدام فیلد Set-Cookie برای نشست سامانهٔ درس مناسب است؟

الف) `sid=7f3a9c; Domain=scu.ac.ir; Secure`

ب) `user=401234567; Max-Age=31536000; HttpOnly`

ج) `__Host-sid=7f3a9c; Path=/; Secure; HttpOnly; SameSite=Lax`

د) `sid=7f3a9c; HttpOnly; SameSite=None`

کدام فیلد Set-Cookie برای نشست سامانهٔ درس مناسب است؟

الف) `sid=7f3a9c; Domain=scu.ac.ir; Secure`

ب) `user=401234567; Max-Age=31536000; HttpOnly`

ج) `__Host-sid=7f3a9c; Path=/; Secure; HttpOnly; SameSite=Lax`

د) `sid=7f3a9c; HttpOnly; SameSite=None`

پاسخ: ج) در گزینهٔ الف، کوکی به همهٔ زیردامنه‌ها فرستاده می‌شود و JavaScript می‌تواند آن را بخواند. در گزینهٔ ب، شمارهٔ دانشجویی به‌جای شناسهٔ تصادفی قرار گرفته و قابل حدس‌زدن است. گزینهٔ د نیز بدون `Secure` پذیرفته نمی‌شود.

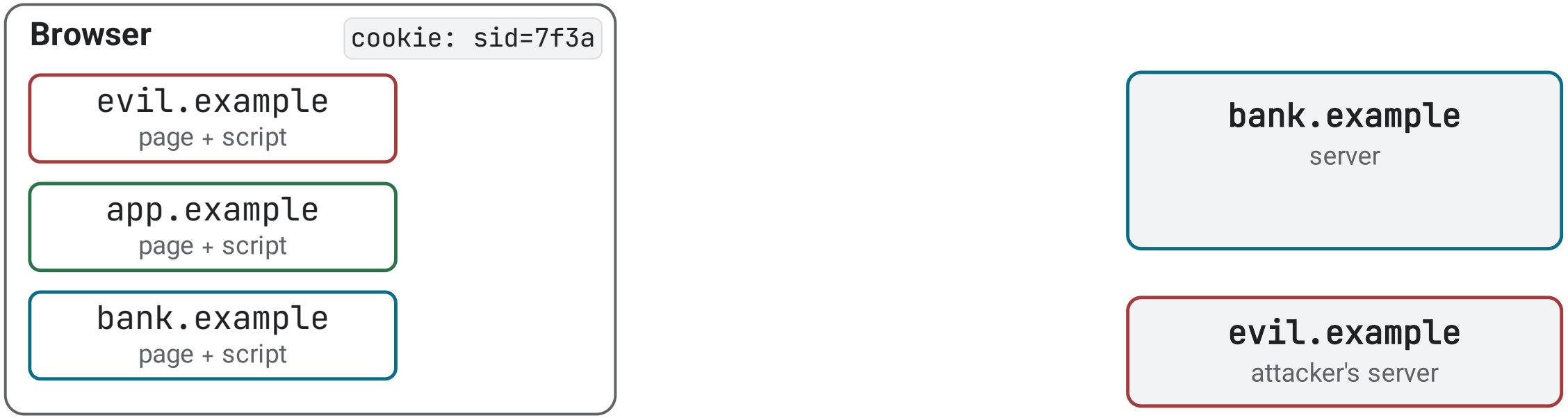
> **هر ویژگی در این خط، به یکی از مسئله‌هایی پاسخ می‌دهد که امروز بررسی کردیم.**

این مفاهیم چه تصمیم‌های مهندسی را بهتر می‌کنند؟

مدل ذهنی	تصمیم مهندسی
سرور فقط مقدار کوکی را می‌بیند	شناسهٔ تصادفی در کوکی `__Host-` با `HttpOnly`
SOP خواندن را می‌بندد، نه فرستادن را	فهرست دقیق origin و کنترل دسترسی در سرور
مرورگر کوکی را برای مقصد می‌فرستد، از هر صفحه‌ای	`SameSite=Lax` و یک دفاع دوم؛ بی‌تغییر با GET
کوکی را مرورگر می‌افزاید، token را کد برنامه	کوکی برای نشست مرورگر، token برای API

جلسهٔ بعد: در مسیر انتقال کوکی، چه کسی می‌تواند آن را ببیند؟ TLS زیر HTTP چه چیزی را عوض می‌کند؟

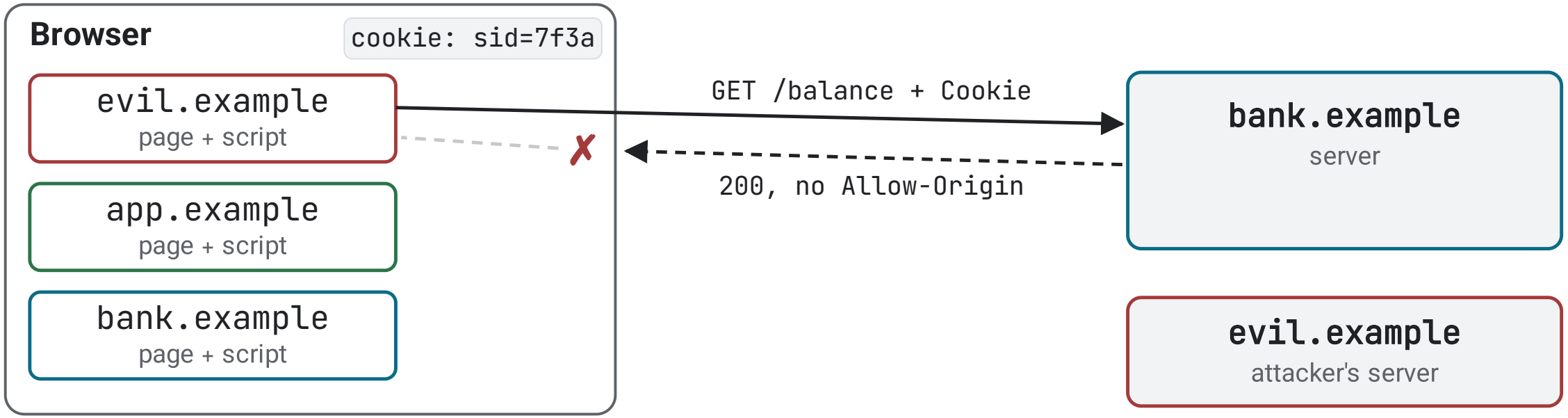
مرور: SOP و CORS و CSRF و XSS روی یک شکل



یک مرورگر، سه صفحه، دو سرور. کوکی نشست `bank.example` در مرورگر است و خودکار همراه هر درخواست به آن می‌رود.

> **پرسش کلیدی: کد چه کسی، در کدام origin، پاسخ را می‌خواند یا فقط درخواست می‌فرستد؟**

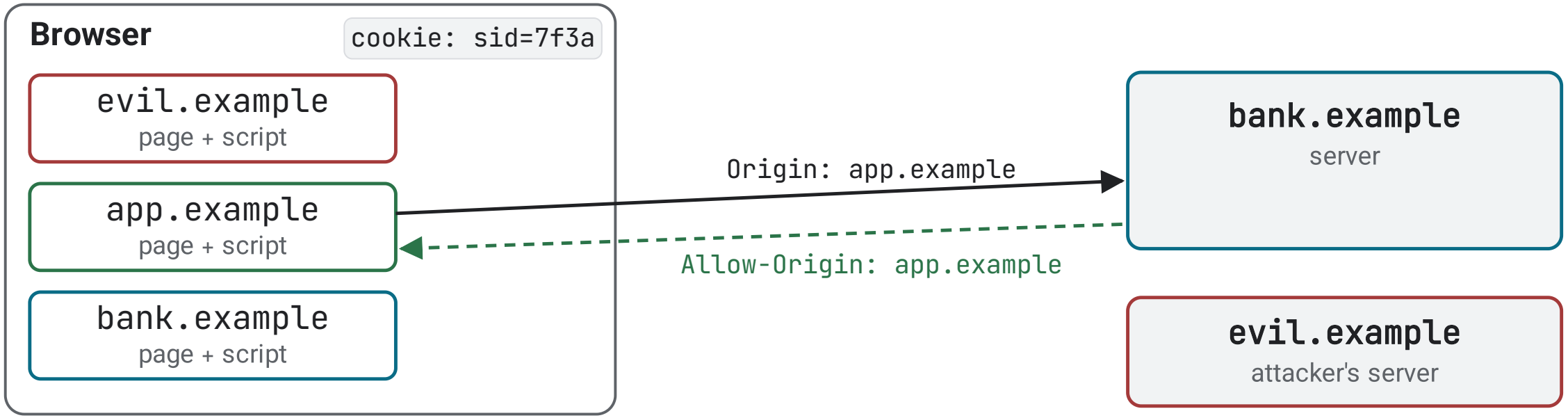
مرور: SOP و CORS و CSRF و XSS روی یک شکل



SOP اسکریپت `evil.example` درخواست می‌فرستد و پاسخ هم می‌رسد؛ اما مرورگر پاسخ را به اسکریپتِ origin دیگر **نمی‌دهد**. SOP خواندن را می‌بندد، نه فرستادن را.

> **پرسش کلیدی: کد چه کسی، در کدام origin، پاسخ را می‌خواند یا فقط درخواست می‌فرستد؟**

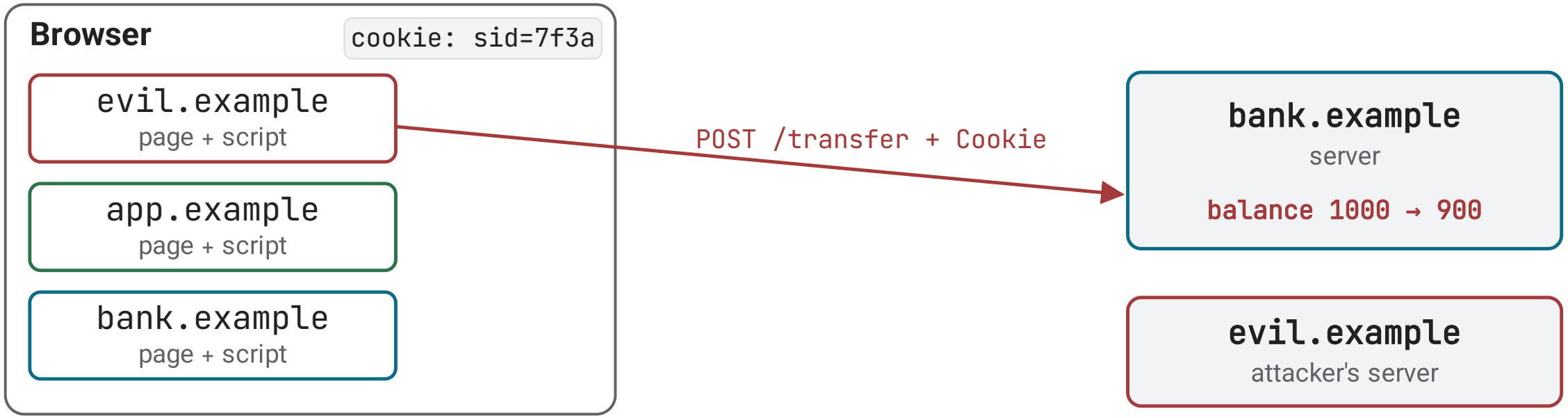
مرور: SOP و CORS و CSRF و XSS روی یک شکل



CORS سرور در پاسخ اعلام می‌کند `app.example` اجازهٔ خواندن دارد؛ مرورگر پاسخ را به اسکریپت همان origin می‌دهد. CORS دری است که سرور در دیوار SOP باز می‌کند.

> **پرسش کلیدی: کد چه کسی، در کدام origin، پاسخ را می‌خواند یا فقط درخواست می‌فرستد؟**

مرور: SOP و CORS و CSRF و XSS روی یک شکل



CSRF صفحهٔ `evil.example` فرم پنهانی به `/transfer` می‌فرستد و کوکی هم خودکار همراهش می‌رود. مهاجم پاسخ را نمی‌خواند؛ **اثر** درخواست کافی است. دفاع: `SameSite` و token ضد CSRF.

> **پرسش کلیدی: کد چه کسی، در کدام origin، پاسخ را می‌خواند یا فقط درخواست می‌فرستد؟**

مرور: SOP و CORS و CSRF و XSS روی یک شکل



XSS کد مهاجم **درون** صفحهٔ `bank.example` اجرا می‌شود و هم‌origin است؛ پس SOP و CORS و `SameSite` جلویش را نمی‌گیرند. صفحه را می‌خواند و به سرور مهاجم می‌فرستد. دفاع: escape خروجی و CSP؛ کوکی `HttpOnly` را نمی‌خواند.

> **پرسش کلیدی: کد چه کسی، در کدام origin، پاسخ را می‌خواند یا فقط درخواست می‌فرستد؟**

مرور سریع: چهار مفهوم امنیت وب

	چیست و چه می‌کند	نقش مرورگر	نقش سرور	محدودیت
SOP	قاعدهٔ مرورگر؛ دسترسی اسکریپت به origin دیگر را محدود می‌کند	**خواندن پاسخ origin دیگر را می‌بندد**	—	ارسال درخواست را لزوماً نمی‌بندد
CORS	سازوکار اشتراک منابع میان originها؛ استثنا بر SOP	اجازه را اعمال می‌کند	**originهای مجاز را اعلام می‌کند**	جلوی رسیدن درخواست و CSRF را نمی‌گیرد
CSRF	حمله **از بیرون**؛ درخواست ناخواسته به نام کاربر	کوکی را طبق `SameSite` می‌فرستد	`SameSite` را تعیین و token ضد CSRF را می‌سنجد	`SameSite` تنها دفاع کامل نیست
XSS	حمله **از درون**؛ کد مهاجم در origin معتبر اجرا می‌شود	CSP را اعمال و کوکی `HttpOnly` را پنهان می‌کند	**خروجی را escape می‌کند** و CSP می‌فرستد	CSP جای رفع آسیب‌پذیری را نمی‌گیرد

نقش اصلی هر مفهوم با پس‌زمینهٔ زرد مشخص شده است.

SOP و CORS مرز خواندن را می‌کشند؛ CSRF و XSS از دو سوی همین مرز می‌آیند.

مرور: state نشست کجا بماند؟



شناسهٔ نشست

خروج و ابطال فوری؛ اما جستجو در هر درخواست و حافظهٔ مشترک میان سرورها

JWT

هر سرور بی‌جستجو می‌سنجد؛ اما ابطال پیش از `exp` دشوار است و محتوا را هر دارنده می‌خواند

پروتکل در همهٔ حالت‌ها stateless است؛ پرسش این است که state نشست نزد کلاینت بماند یا سرور.

مرور: درون یک JWT چیست؟

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI0MDEyMzQ1NjciLCJyb2xlIjoic3R1ZGVudCIsImlhdCI6MTc5MjQ2NzAwMCwiZXhwIjoxNzkyNDY3OTAwfQ.j3n6IS2NIA4p2tRsvxD1_F1yjiIhn1sd-OQdq0VcCBs
```

header

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

الگوریتم امضا

payload

```
{
  "sub": "401234567",
  "role": "student",
  "iat": 1792467000,
  "exp": 1792467900
}
```

ادعاها؛ exp یعنی ۱۵ دقیقه عمر

signature

```
HMAC-SHA256(
  header + "." + payload,
  server key
)
```

فقط دارندهٔ کلید می‌سازد

- هر بخش **Base64url** است، نه رمز؛ امضا فقط جلوی تغییر را می‌گیرد. جزئیات در فصل بعد

> **JWT خواناست اما تغییرناپذیر؛ راز را در payload نگذارید.**

تمرین ۳: پیاده‌سازی ورود با کوکی و بررسی CSRF

چالش

روی سرور محلی خود، ورود مبتنی بر کوکی نشست را پیاده‌سازی کنید. سپس از یک origin دیگر، با یک فرم پنهان به نام کاربرِ واردشده انتقال پول انجام دهید و در مرحلهٔ بعد جلوی این حمله را بگیرید.

سؤال

کوکی `SameSite=Strict` است و سرور شما روی `http://localhost:8000` . صفحهٔ مهاجم روی `http://localhost:9000` موفق می‌شود؟ و روی `http://127.0.0.1:9000` ؟ چرا؟

پل

در این تمرین، کوکی از طریق HTTP و بدون رمزنگاری منتقل می‌شود. در جلسهٔ ۴ بررسی می‌کنیم چه کسی می‌تواند آن را ببیند و TLS چه چیزی را تغییر می‌دهد.

تا جمعه ۸ آبان، ۲۳:۵۹

ie1405-scu.github.io/exercises/02-socket-to-tls/

نقشهٔ فصل ۱: کجای راهیم؟

ch1/syllabus.todo

- [x] جلسهٔ ۱: از URL تا سرور — URL و Origin و DNS، و TLS از دور
- [x] جلسهٔ ۲: یک تراکنش HTTP — متد و کد وضعیت و header
- [>] **جلسهٔ ۳: state و هویت — cookie و CORS و CSRF** ▶
- [ ] جلسهٔ ۴: زیربنای HTTP — TCP و QUIC و TLS
- [ ] جلسهٔ ۵: واسطه‌ها — cache و proxy و CDN
- [ ] جلسهٔ ۶: وقتی سرور اول حرف می‌زند — SSE و WebSocket
- [ ] جلسهٔ ۷: کلاینت‌های خودکار — agent و MCP

هر جلسه یک حلقه از همان زنجیرهٔ ابتدای فصل را بررسی می‌کند.

برای مطالعه و کنجکاوی بیشتر (۱ از ۲)

۱ کوکی `Partitioned` چه مشکلی از کوکی شخص ثالث را حل می‌کند، و چرا پس از بازنشستگی Privacy Sandbox ماند؟

سرنخ: دربارهٔ CHIPS بخوانید؛ کلید کوکی به سایت سطح بالا هم وابسته می‌شود.

۲ چرا `fetch` با `mode: 'no-cors'` پاسخی دریافت می‌کند که نمی‌توان بدنهٔ آن را خواند؟

سرنخ: «opaque response» در استاندارد Fetch.

۳ چرا اسکریپت صفحه نمی‌تواند مقدار فیلد `Sec-Fetch-Site` یا `Origin` را جعل کند؟

سرنخ: «forbidden request header» در استاندارد Fetch؛ پیشوند `Sec-` برای همین ساخته شد.

برای مطالعه و کنجکاوی بیشتر (۲ از ۲)

۴ **اگر سایت شما روی `ali.github.io` میزبانی شود، مرورگر با کوکی `Domain=github.io` چه برخوردی می‌کند؟**

سرنخ: بخش PRIVATE در Public Suffix List؛ با DevTools امتحانش کنید.

۵ **چرا مرورگرها پس از حملهٔ Spectre، سندهای متعلق به siteهای مختلف را در پردازه‌های جداگانه اجرا می‌کنند؟ SOP بدون این جداسازی چه محدودیتی داشت؟**

سرنخ: Site Isolation در Chromium، و header `Cross-Origin-Opener-Policy`.

۶ **نشستی که به کلیدی درون دستگاه پیوند خورده است، در برابر بدافزاری که روی همان دستگاه اجرا می‌شود از چه چیزهایی محافظت می‌کند و در برابر چه چیزهایی بی‌دفاع است؟**

سرنخ: مشخصات Device Bound Session Credentials در W3C؛ فرق «دزدیدن کوکی» با «استفاده از کوکی در همان دستگاه».

منابع

- منبع درسی: کتاب *Computer Networking* نوشتهٔ Kurose و Ross، بخش ۲.۲.۴
- RFC 6265: Cookies · 6265bis draft
- MDN: Set-Cookie
- MDN: CORS · WHATWG Fetch
- RFC 9110 §11: HTTP Authentication · RFC 6750
- RFC 6749: OAuth 2.0 · RFC 9728
- W3C: WebAuthn
- MDN: CSRF · OWASP
- Cloudflare: HAR files and the Okta breach
- The Hacker News: DBSC in Chrome 146
- Privacy Sandbox: next steps · update
- PortSwigger: CORS misconfigurations
- Zeller and Felten: CSRF
- Chromium: SameSite rollback · FAQ
- Brave: prompt injection in Comet
- MCP: Authorization